

高级搜索树

红黑树： 动机

并发性: Concurrent Access To A Database

❖ 修改之前先**加锁** (lock) ; 完成后**解锁** (unlock)

访问延迟主要取决于 “lock/unlock” 周期

❖ 对于BST而言, 每次修改过程中, 唯

结构有变 (reconstruction) 处才需加锁

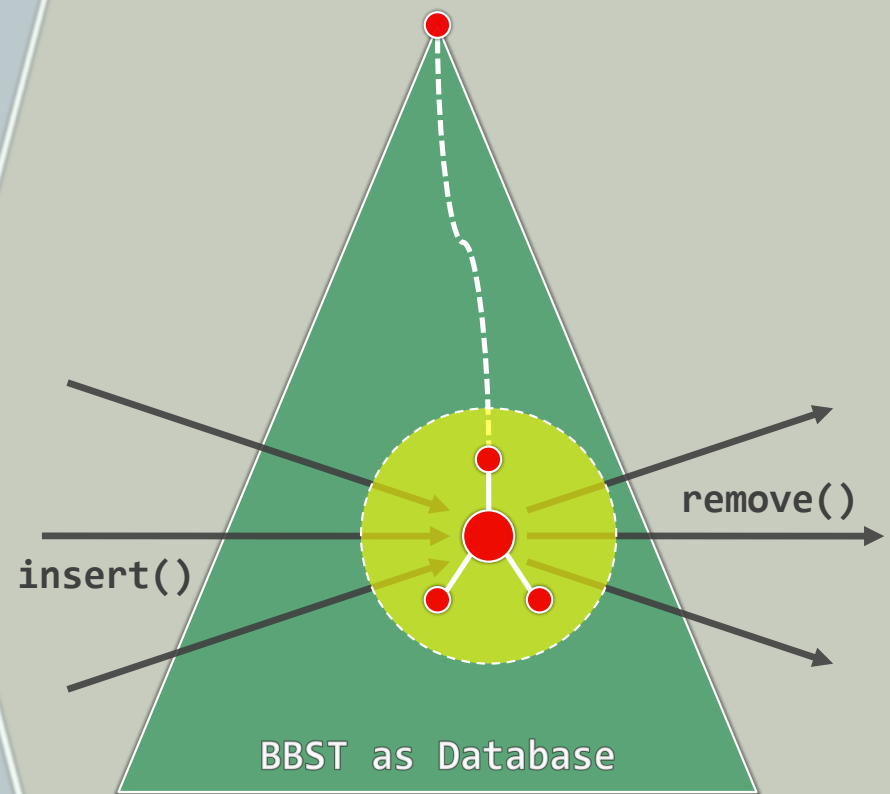
访问延迟主要取决于这类局部之**数量**...

❖ Splay: 结构变化剧烈, 最差可达 $\mathcal{O}(n)$

❖ AVL: `remove()` 时 $\mathcal{O}(\log n)$

——尽管 `insert()` 时可保证 $\mathcal{O}(1)$

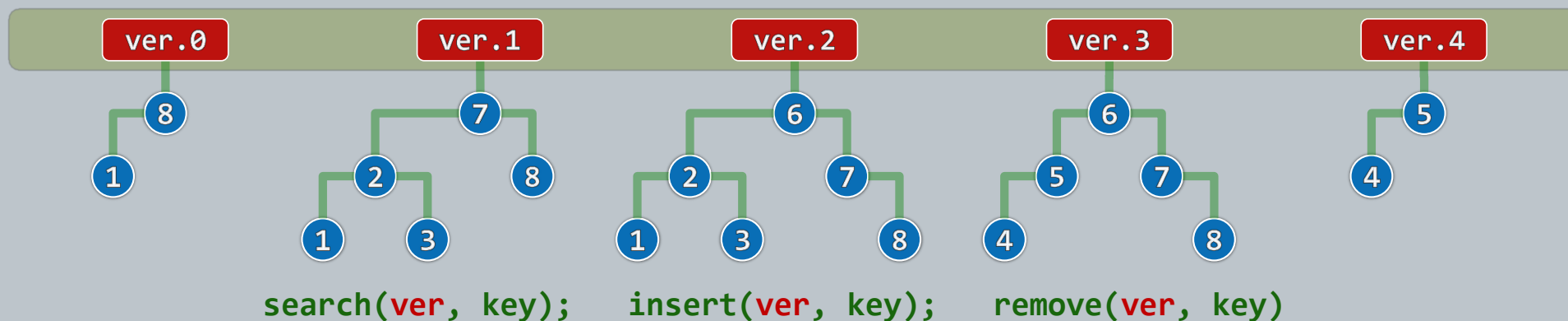
❖ Red-Black: 无论 `insert/remove`, **均不超过** $\mathcal{O}(1)$



持久性

❖ Persistent structures: 支持对**历史**版本的访问

//ephemeral



❖ 蛮力实现: 每个版本独立保存; 各版本自成一个搜索结构

❖ 单次操作 $\mathcal{O}(\log h + \log n)$, 累计 $\mathcal{O}(n \cdot h)$ 时间/空间

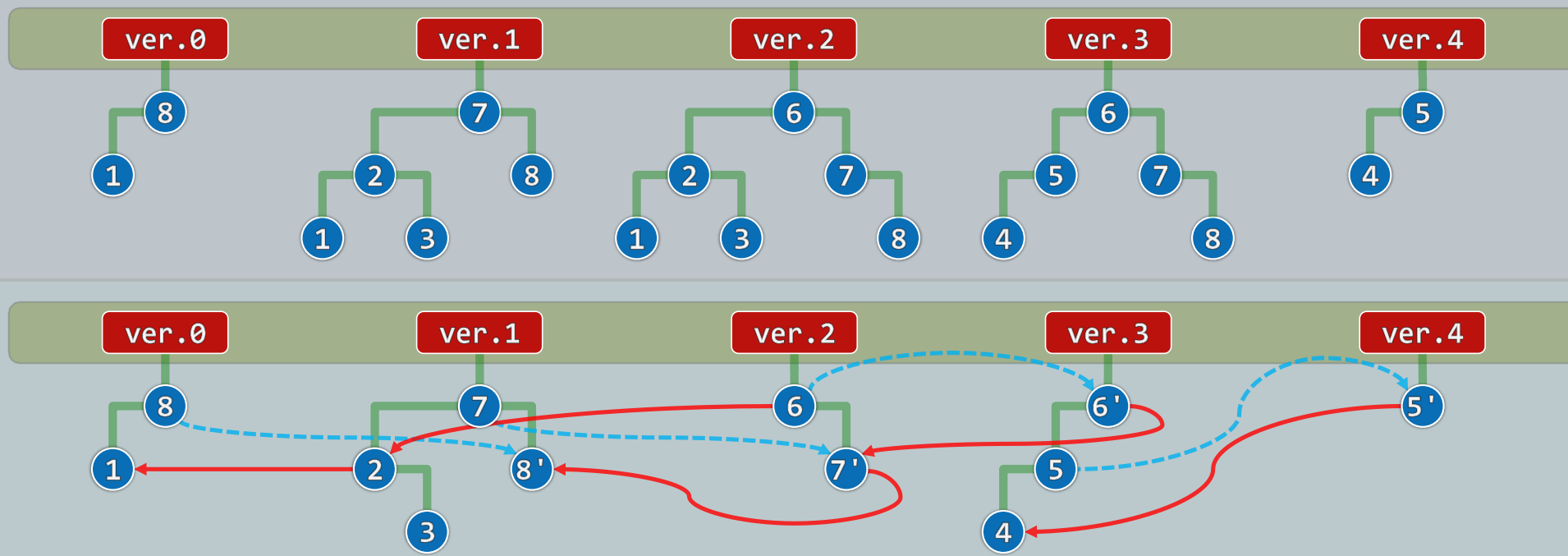
//h = |history|

❖ 挑战: 可否将复杂度控制在 $\mathcal{O}(n + h \cdot \log n)$ 以内?

❖ 可以! 为此需利用相邻版本之间的**相关性**...

压缩存储

❖ 大量共享，少量更新：每个版本的新增复杂度，仅为 $\mathcal{O}(\log n)$



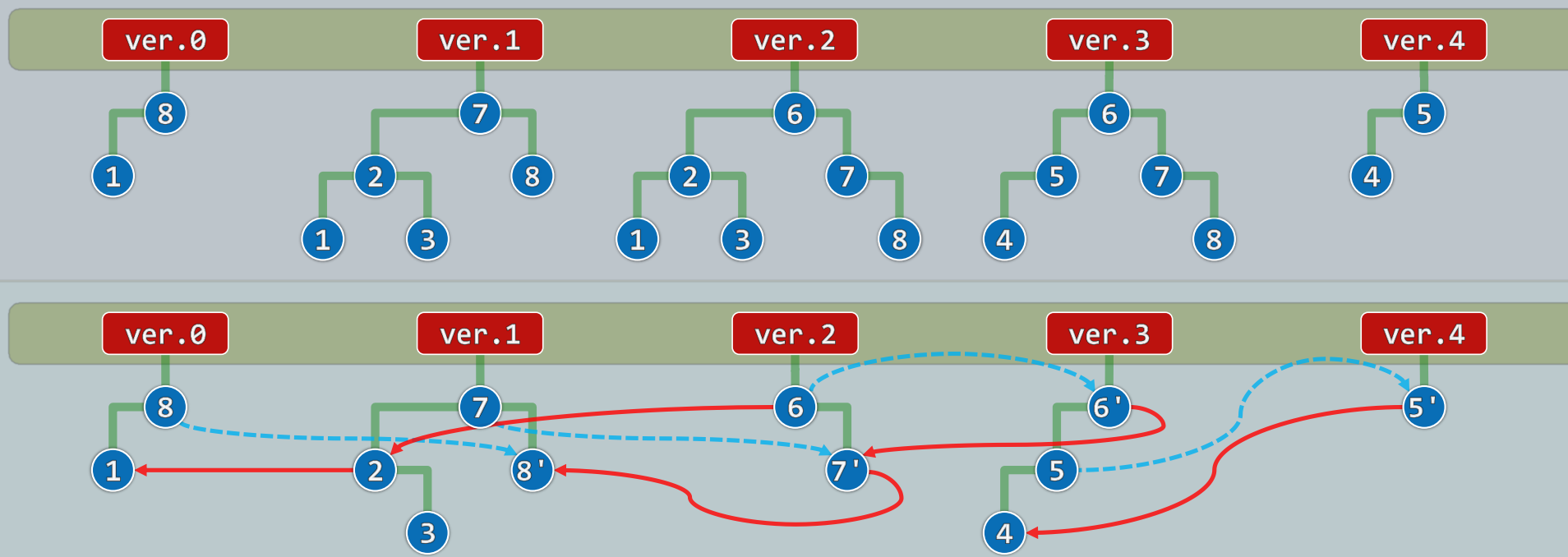
❖ **Partial Persistence:** 仅支持对历史版本的**读取**

// 监控录像、飞行器黑盒子，等等

❖ 这类情况下，还可进一步提高至总体 $\mathcal{O}(n + h)$ 、单版本 $\mathcal{O}(1)$...

$O(1)$ 重构

❖ 为此，就树形结构的**拓扑**而言，相邻版本之间的差异不能超过 $O(1)$



❖ 很遗憾，AVL、Splay等BBST均不具备这一性质；须另辟蹊径...

高级搜索树

红黑树：结构

红与黑

❖ 1972, [R. Bayer](#)

Symmetric Binary B-Tree

❖ 1978, [L. Guibas](#) & [R. Sedgwick](#)

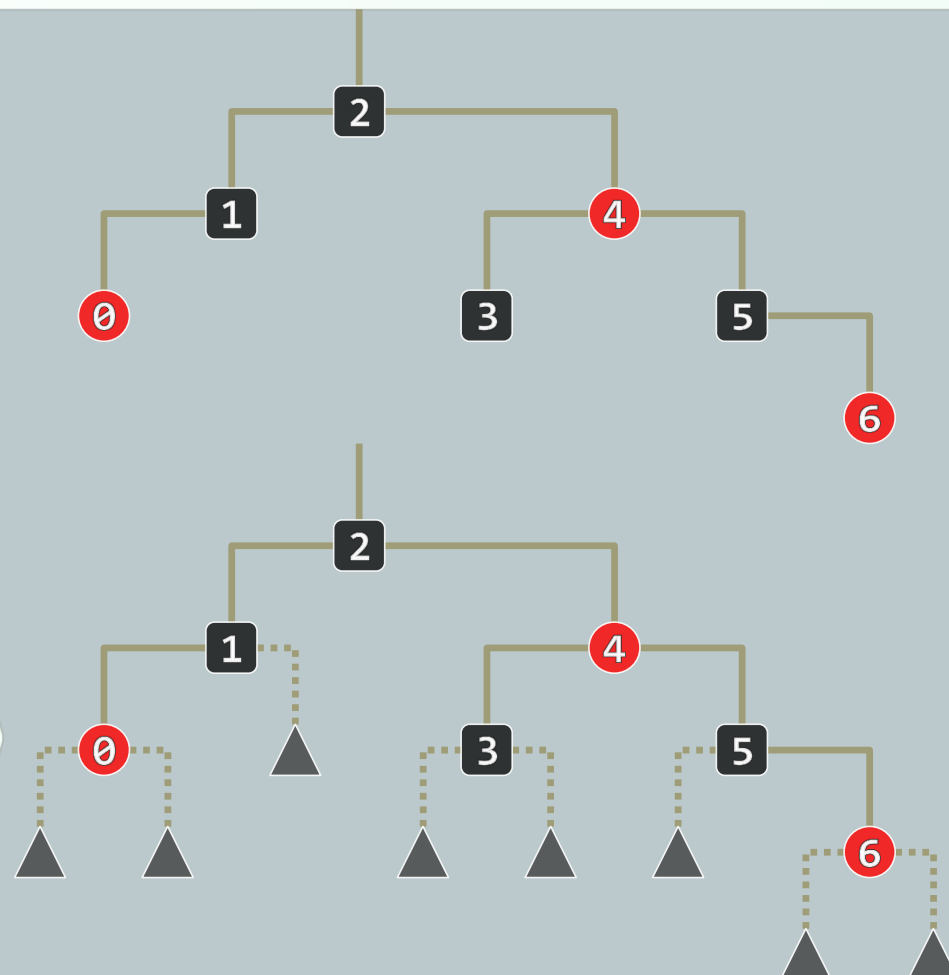
Red-Black Tree

❖ 1982, H. Olivie

Half-Balanced Binary Search Tree

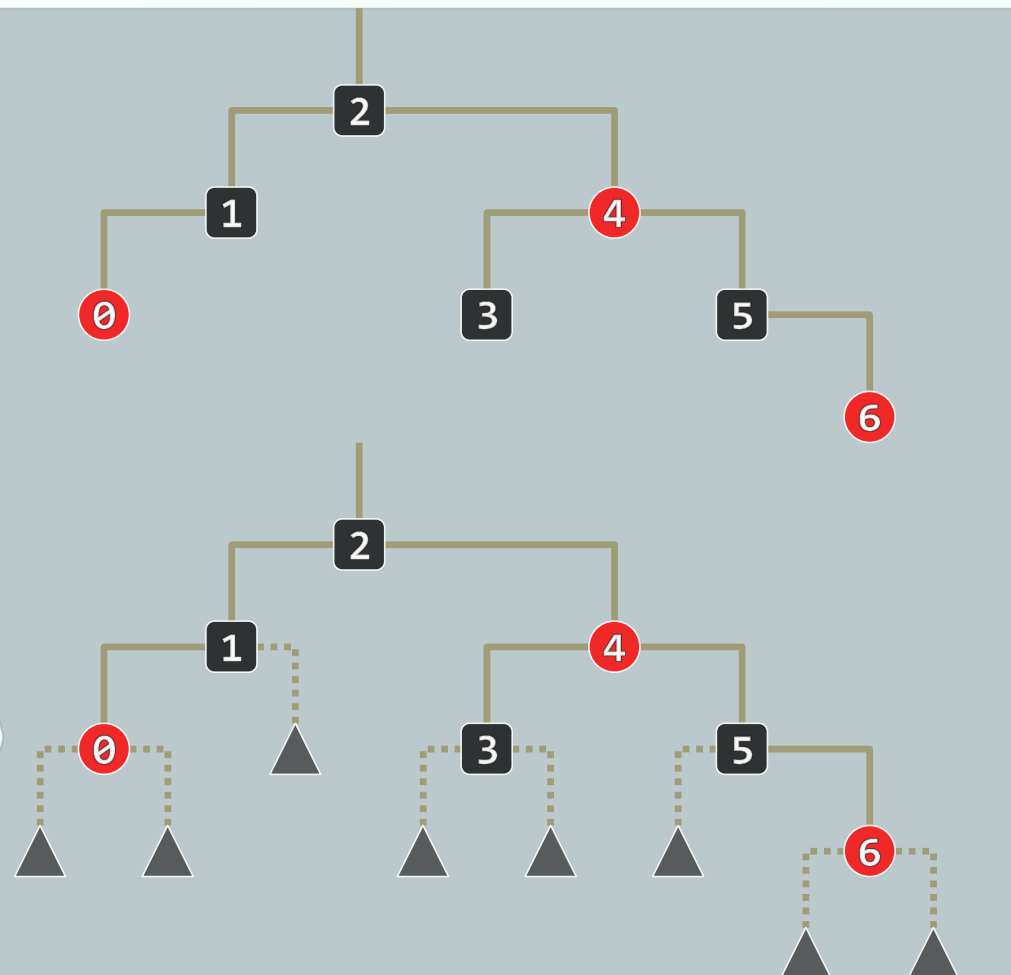
❖ 由红、黑两类节点组成的[BST](#)

统一增设外部节点NULL, 使之成为**真二叉树**



规则

- 1) 树根：必为黑色
 - 2) 外部节点：均为黑色
 - 3) 红节点：只能有黑孩子（及黑父亲）
 - 4) 外部节点：黑深度（黑的真祖先数目）相等
 - 亦即根（全树）的黑高度
 - 子树的黑高度，即后代NULL的相对黑深度
- ❖ 节点的颜色，只能显式地记录？
- ❖ 以上定义颇为费解，有直观解释吗？
- 如此定义的BST，也是BBST？



提升与折叠：红黑树 = (2,4)-树

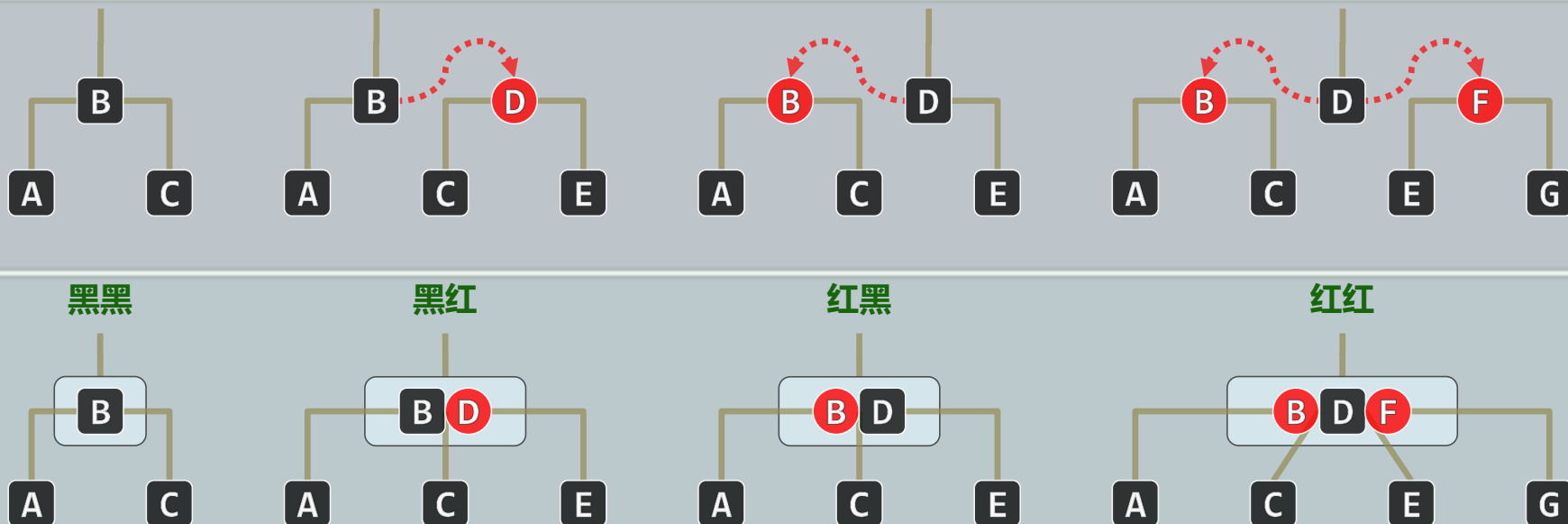
❖ 将红节点**提升**至与其（黑）父亲等高——于是每棵红黑树，都对应于一棵(2,4)-树

❖ 将**黑节点**与其**红孩子**视作关键码，再合并为B-树的**超级节点**...

//元音 + 辅音 = 音节

❖ 无非四种组合，分别对应于**4阶B-树**的一类内部节点

//反过来呢?



红黑树 $\in$ BBST

- ❖ 包含 n 个内部节点的红黑树 T , 高度 $h = \mathcal{O}(\log n)$ //既然B-树是平衡的, 由等价性红黑树也应是

$$\log_2(n+1) \leq h \leq 2 \cdot \log_2(n+1)$$

- ❖ 若 T 高度为 h , 红/黑高度为 R/H , 则

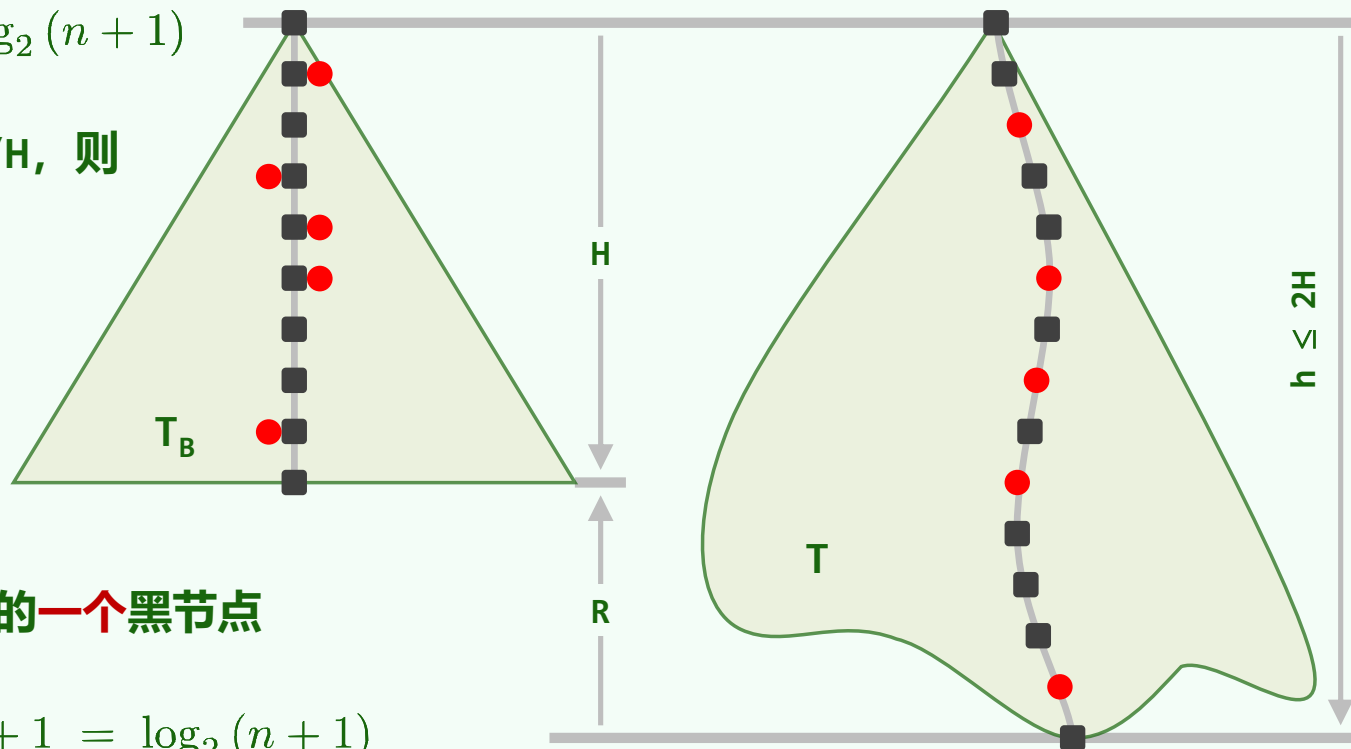
$$H \leq h \leq R + H \leq 2 \cdot H$$

- ❖ 若 T 所对应的B-树为 T_B

则 H 即是 T_B 的高度

- ❖ T_B 的每个节点, 都恰好包含 T 的一个黑节点

- ❖ 于是, $H \leq \log_{[4/2]} \frac{n+1}{2} + 1 = \log_2(n+1)$



RedBlack

```
template <typename T> class RedBlack : public BST<T> { //红黑树
public:      //BST::search()等其余接口可直接沿用
           BinNodePosi<T> insert( const T & e ); //插入 (重写)
           bool remove( const T & e ); //删除 (重写)
protected: void solveDoubleRed( BinNodePosi<T> x ); //双红修正
            void solveDoubleBlack( BinNodePosi<T> x ); //双黑修正
            int updateHeight( BinNodePosi<T> x ); //更新节点x的高度 (重写)
};

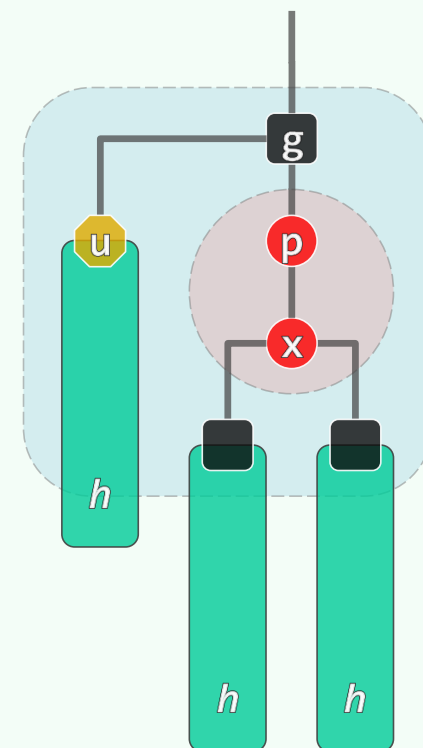
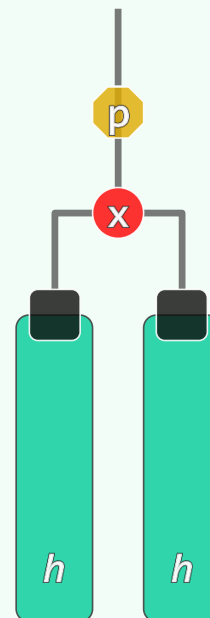
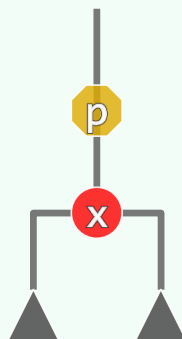
template <typename T> int RedBlack<T>::updateHeight( BinNodePosi<T> x )
{ return x->height = IsBlack( x ) + max( stature( x->lc ), stature( x->rc ) ); }
```

高级搜索树

红黑树：插入

算法

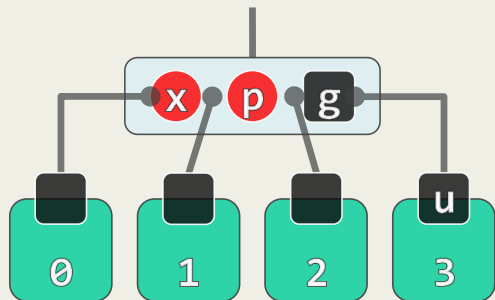
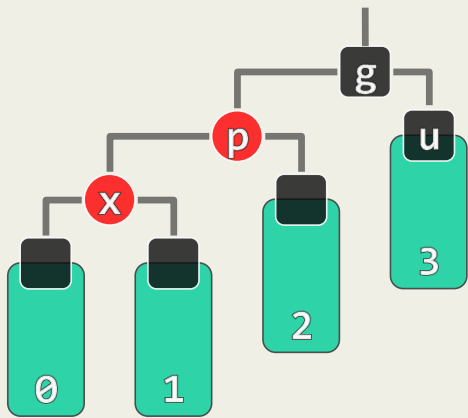
- ❖ 按BST规则插入关键码e `//x = insert(e)`必为叶节点
- ❖ 除非系首个节点（根），x的父亲p = x->parent必存在
首先将x染红 `//x->color = isRoot(x) ? B : R`
- ❖ 至此，条件1、2、4依然满足；
但3不见得，有可能...
- ❖ 双红 (double-red)
`//p->color == x->color == R`
- ❖ 考查：祖父g = p->parent
叔父u = uncle(x) = sibling(p)
- ❖ 以下，视u的颜色，分两种情况处理...



实现

```
❖ template <typename T> BinNodePosi<T> RedBlack<T>::insert( const T & e ) {  
  
    // 确认目标节点不存在（留意对_hot的设置）  
  
    BinNodePosi<T> & x = search( e ); if ( x ) return x;  
  
    // 创建红节点x, 以_hot为父, 黑高度 = 0  
  
    x = new BinNode<T>( e, _hot, NULL, NULL, 0 ); _size++;  
  
    // 如有必要, 需做双红修正, 再返回插入的节点  
  
    BinNodePosi<T> xOld = x; solveDoubleRed( x ); return xOld;  
  
} //无论原树中是否存有e, 返回时总有x->data == e
```

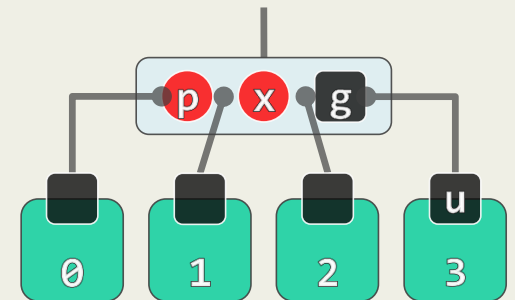
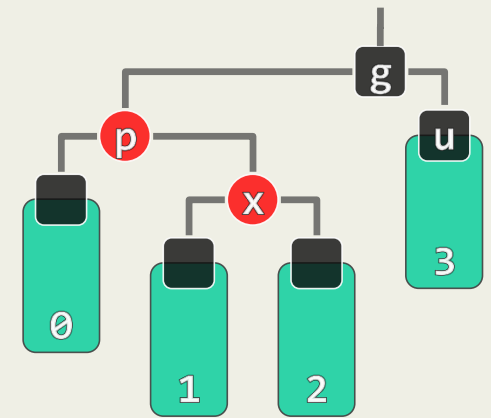
RR-1: $u \rightarrow \text{color} == B$



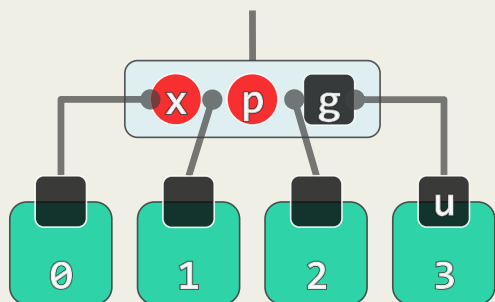
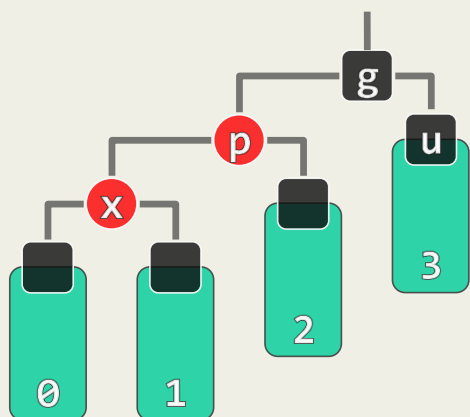
❖ 此时, x 、 p 、 g 的四个孩子
(可能是外部节点)

- 全为黑, 且
- 黑高度相同

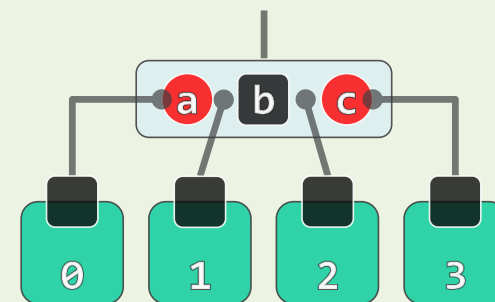
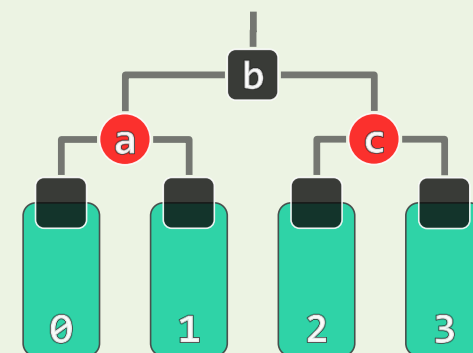
❖ 另两种对称情况, 自行补充



RR-1: $u \rightarrow \text{color} == B$



- ❖ 局部“3+4”重构
b转黑, a或c转红
- ❖ 从B-树的角度, 如何理解?
所谓“非法”, 无非是...
- ❖ 在某三叉节点中插入红关键码后
原黑关键码不再居中 (RRB或BRR)
- ❖ 调整的效果, 无非是
将三个关键码的颜色改为RBR
- ❖ 如此调整, 一蹴而就



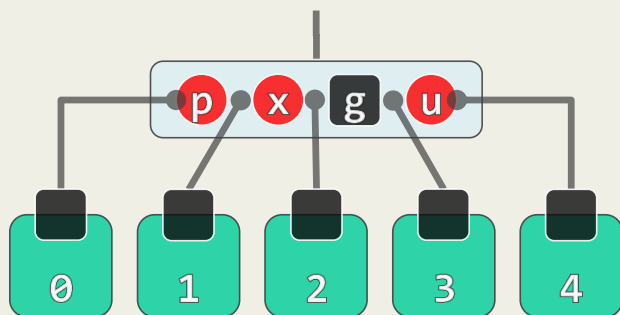
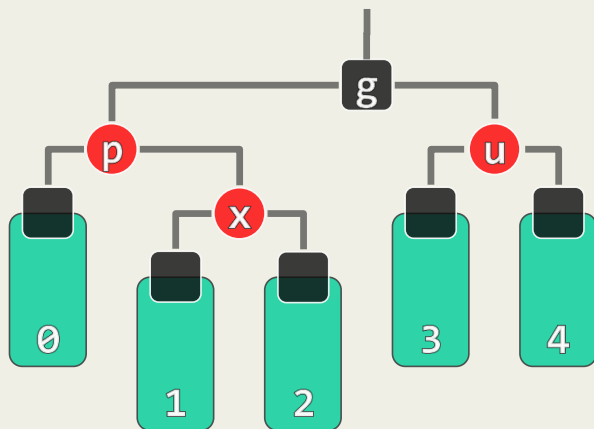
双红修正

```
❖ template <typename T> void RedBlack<T>::solveDoubleRed( BinNodePosi<T> x ) {  
    if ( IsRoot( *x ) ) { //若已（递归）转至树根，则将其转黑，整树黑高度也随之递增  
        { _root->color = RB_BLACK; _root->height++; return; } //否则...  
  
    BinNodePosi<T> p = x->parent; //考查x的父亲p（必存在）  
    if ( IsBlack( p ) ) return; //若p为黑，则可终止调整；否则  
  
    BinNodePosi<T> g = p->parent; //x祖父g必存在，且必黑  
    BinNodePosi<T> u = uncle( x ); //以下视叔父u的颜色分别处理  
  
    if ( IsBlack( u ) ) { /* ... u为黑（或NULL） ... */ }  
    else { /* ... u为红 ... */ }  
  
}
```

RR-1: 实现

```
❖ template <typename T> void RedBlack<T>::solveDoubleRed( BinNodePosi<T> x ) {  
    /* ..... */  
    if ( IsBlack( u ) ) { //u为黑或NULL  
        // 若x与p同侧，则p由红转黑，x保持红；否则，x由红转黑，p保持红  
        if ( IsLChild( *x ) == IsLChild( *p ) ) p->color = RB_BLACK;  
        else x->color = RB_BLACK;  
  
        g->color = RB_RED; //g必定由黑转红  
  
        BinNodePosi<T> gg = g->parent; //great-grand parent  
        BinNodePosi<T> r = FromParentTo( *g ) = rotateAt( x );  
        r->parent = gg; //调整之后的新子树，需与原曾祖父联接  
    } else { /* ... u为红 ... */ }  
}
```

RR-2: $u \rightarrow \text{color} == R$

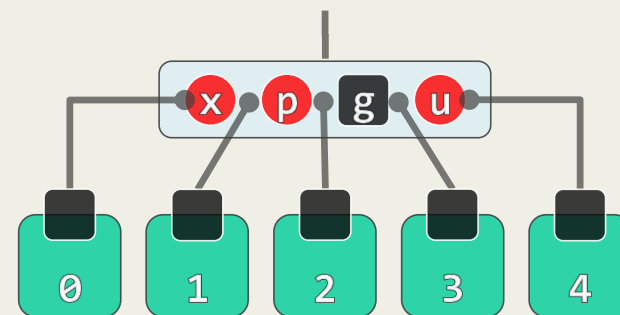
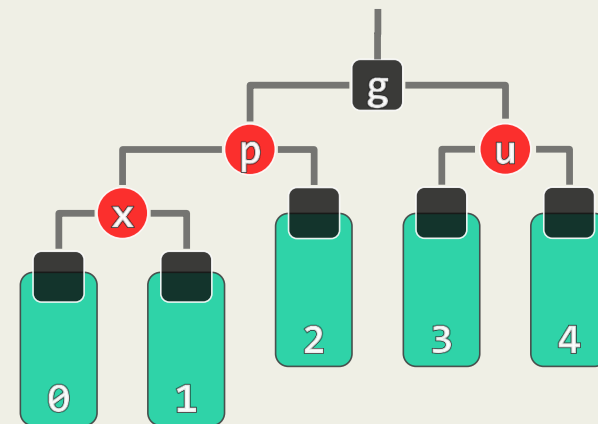


❖ 在B-树中，等效于

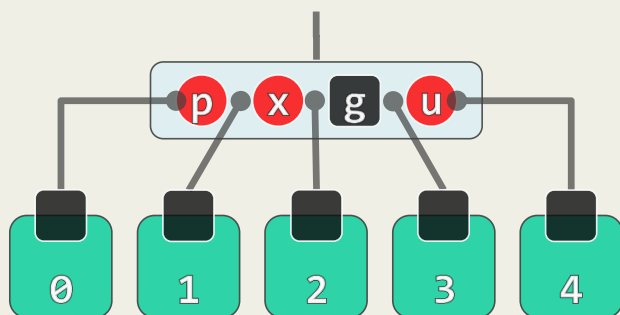
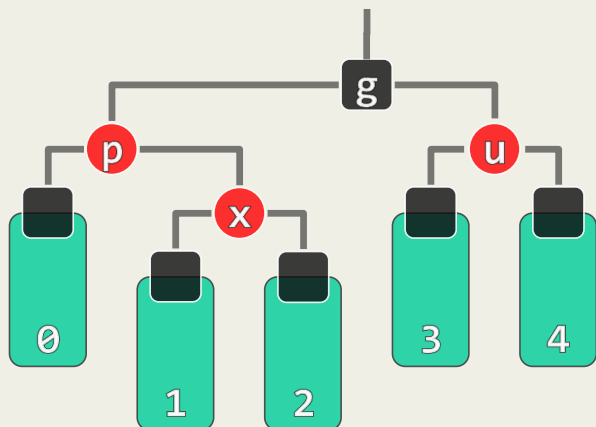
超级节点发生上溢

❖ 另两种对称情况

请自行补充



RR-2: $u \rightarrow \text{color} == R$

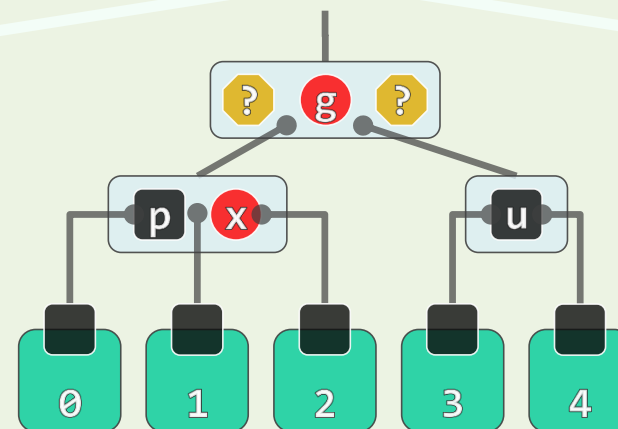
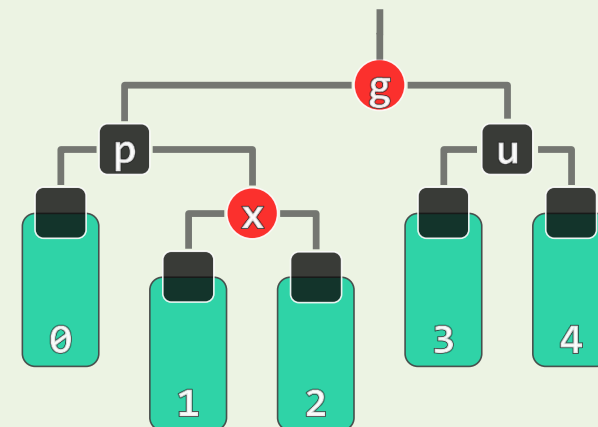


❖ p与u转黑, g转红

在B-树中, 等效于...

❖ 节点分裂

关键码g上升一层



RR-2: $u \rightarrow \text{color} == R$

❖ 既然是分裂，也应有**可能**继续向上传递——亦即， g 与 $\text{parent}(g)$ **再次构成双红**

❖ 果真如此，可：等效地将 g 视作新插入的节点

区分以上两种情况，如法处置

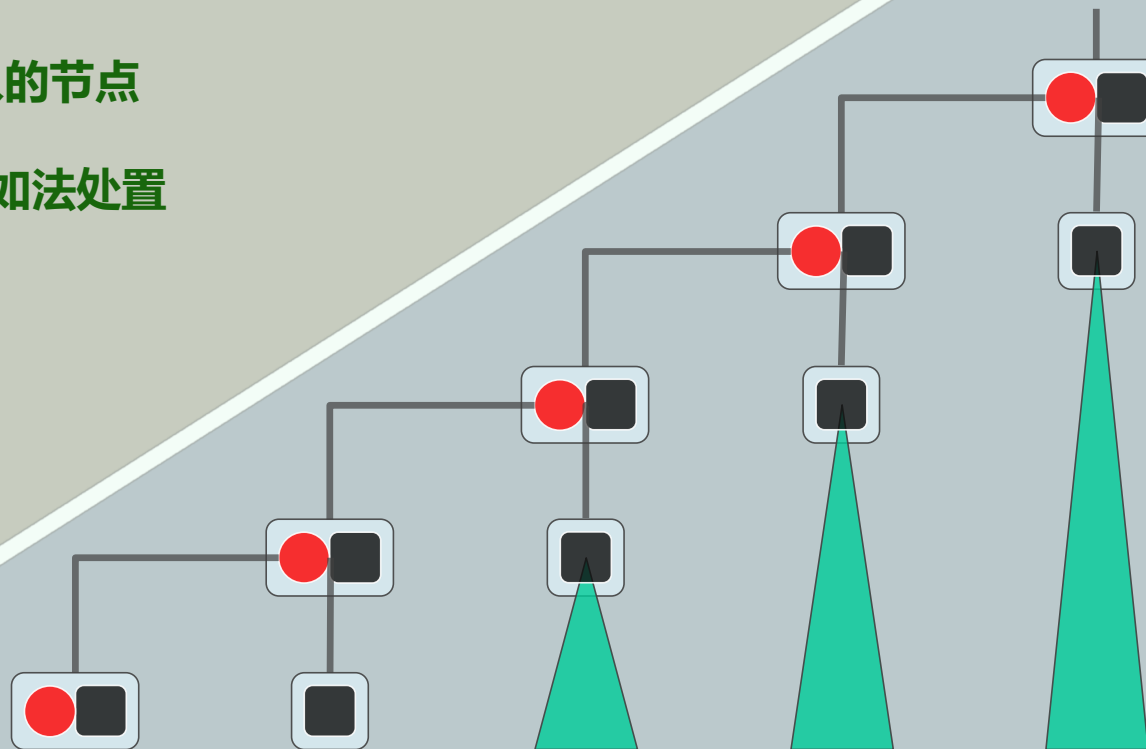
❖ 直到所有条件满足（即不再双红）

或者抵达树根

❖ g 若果真到达树根，则

强行将其转为黑色

（整树黑高度加一）



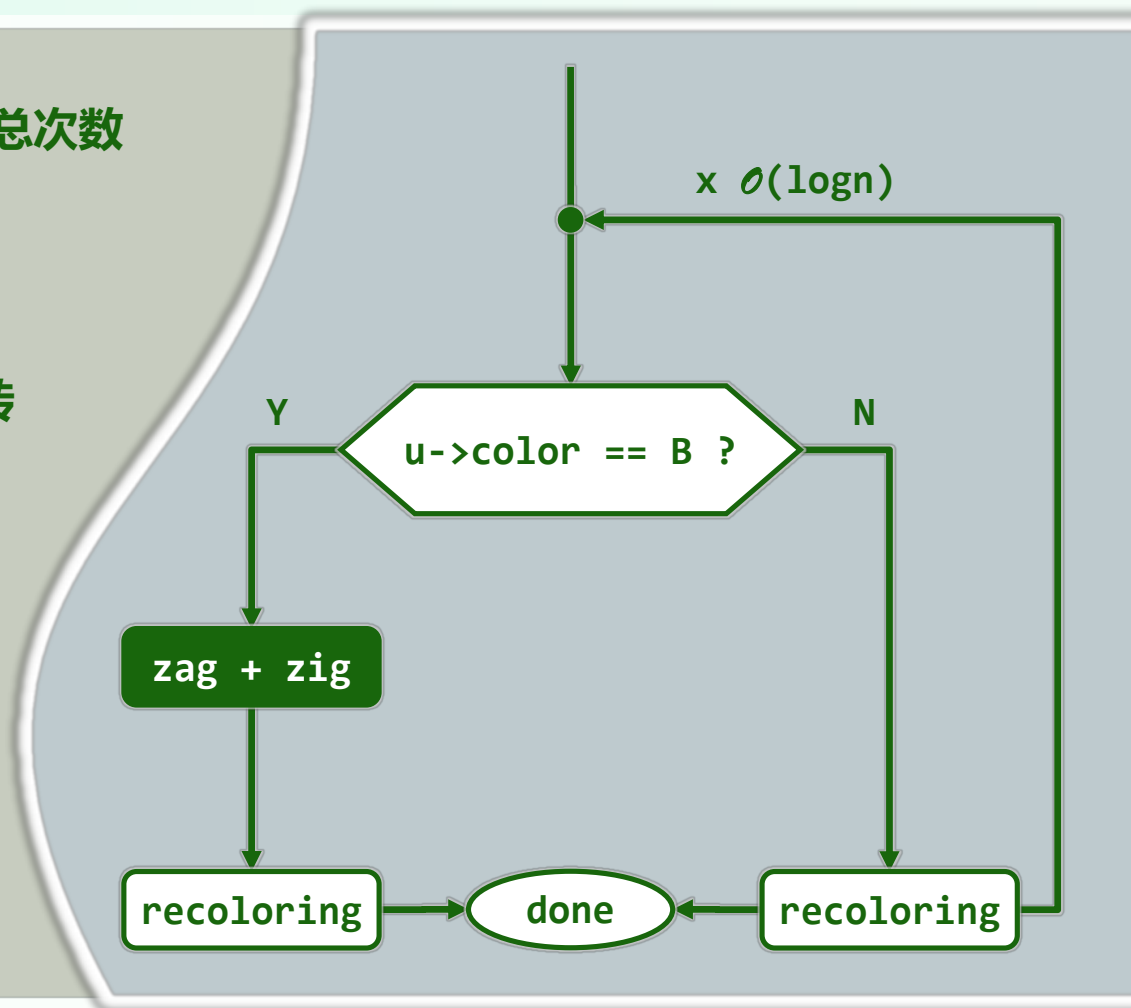
RR-2: 实现

```
❖ template <typename T> void RedBlack<T>::solveDoubleRed( BinNodePosi<T> x ) {  
    /* ..... */  
    if ( IsBlack( u ) ) { /* ... u为黑 (含NULL) ... */ }  
    else { //u为红色  
        p->color = RB_BLACK; p->height++; //p由红转黑, 增高  
        u->color = RB_BLACK; u->height++; //u由红转黑, 增高  
        if ( ! IsRoot( *g ) ) g->color = RB_RED; //g若非根则转红  
        solveDoubleRed( g ); //继续调整g (类似于尾递归, 可优化)  
    }  
}
```

复杂度

- ❖ 重构、染色均只需常数时间，故只需统计其总次数
- ❖ `RedBlack::insert()` 仅需 $O(\log n)$ 时间
- ❖ 其间至多做 $O(\log n)$ 次重染色、 $O(1)$ 次旋转

	旋转	染色	此后
u为黑	1~2	2	调整随即完成
u为红	0	3	可能再次双红 但必上升两层



高级搜索树
红黑树：删除

等效删除

❖ 首先按照BST常规算法，执行

$r = \text{removeAt}(x, \text{_hot})$

// 实际被摘除的可能是x的前驱或后继w

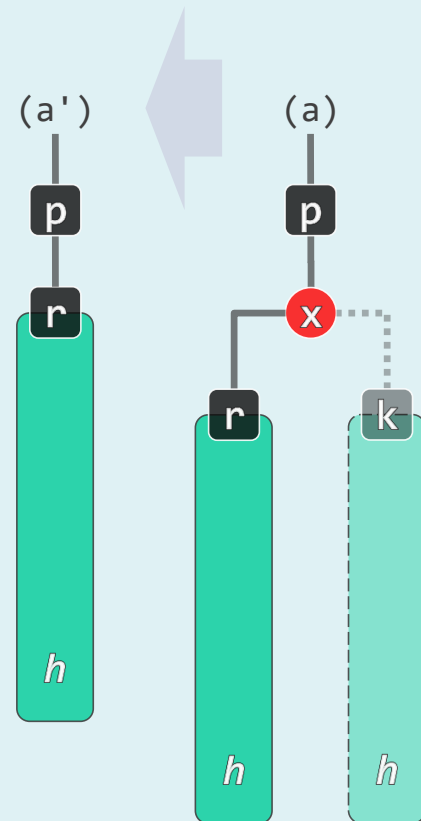
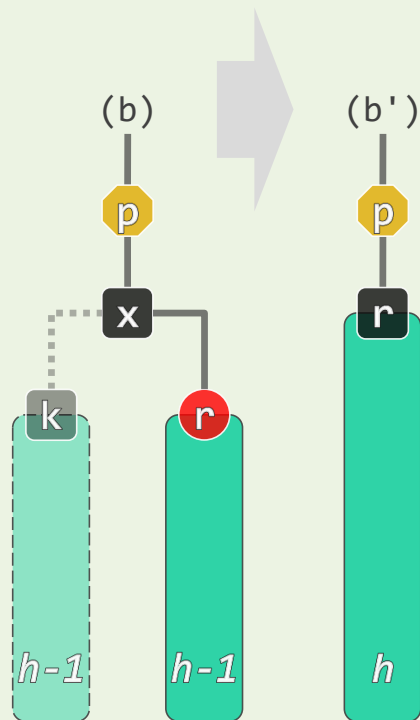
// 简捷起见，以下不妨统称作x

❖ x由孩子r接替，此时另一孩子k必为NULL

❖ 但在随后的调整过程中，x可能逐层上升

❖ 故需假想地、统一地、等效地理解为：

- k为一棵黑高度与r相等的子树，且
- 随x一并摘除（尽管实际上从未存在过）



其一为红

❖ 完成`removeAt()`之后

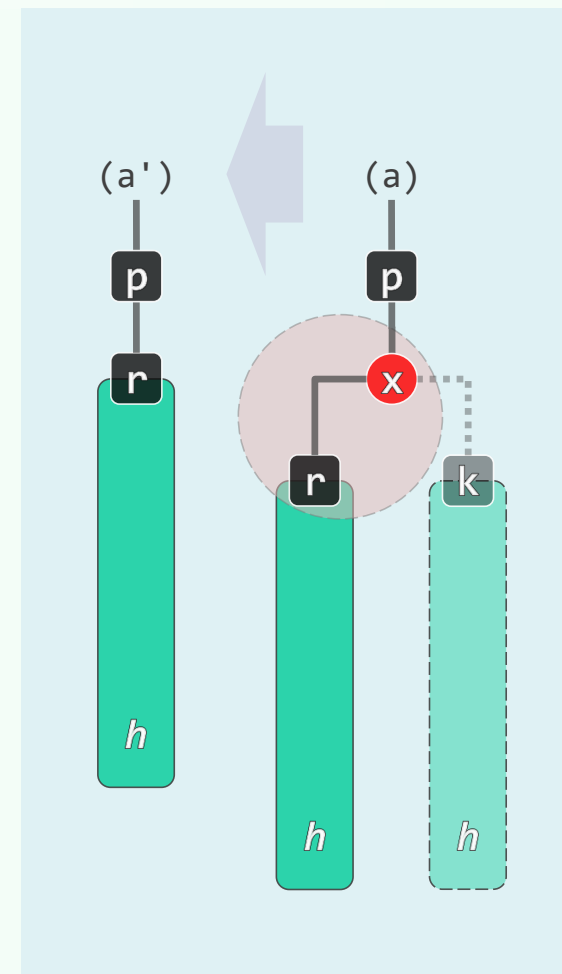
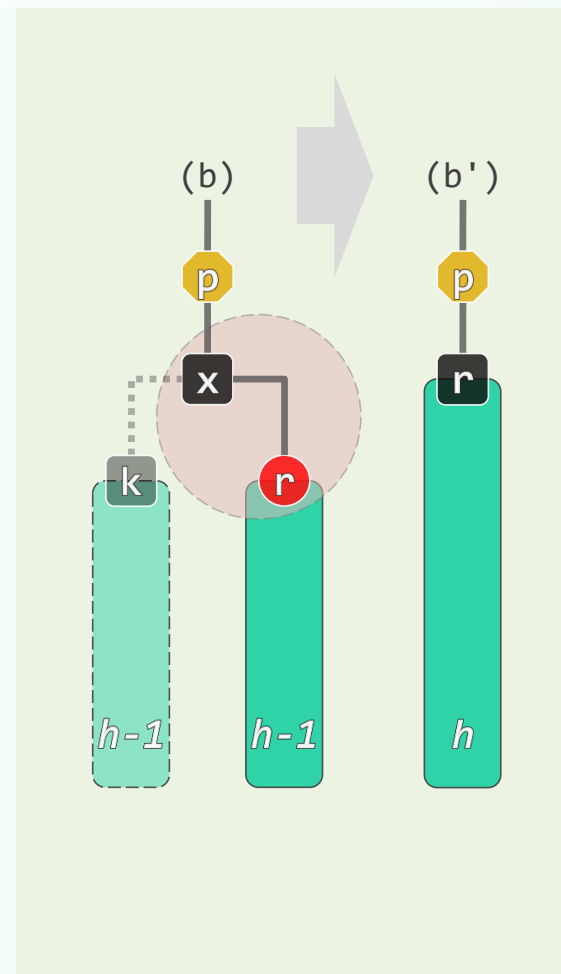
- 条件1、2依然满足
- 但条件3、4却不见得

❖ 在原树中，考查x与r

- 若x为红，则条件3、4自然满足
- 若r为红，则令其与x交换颜色

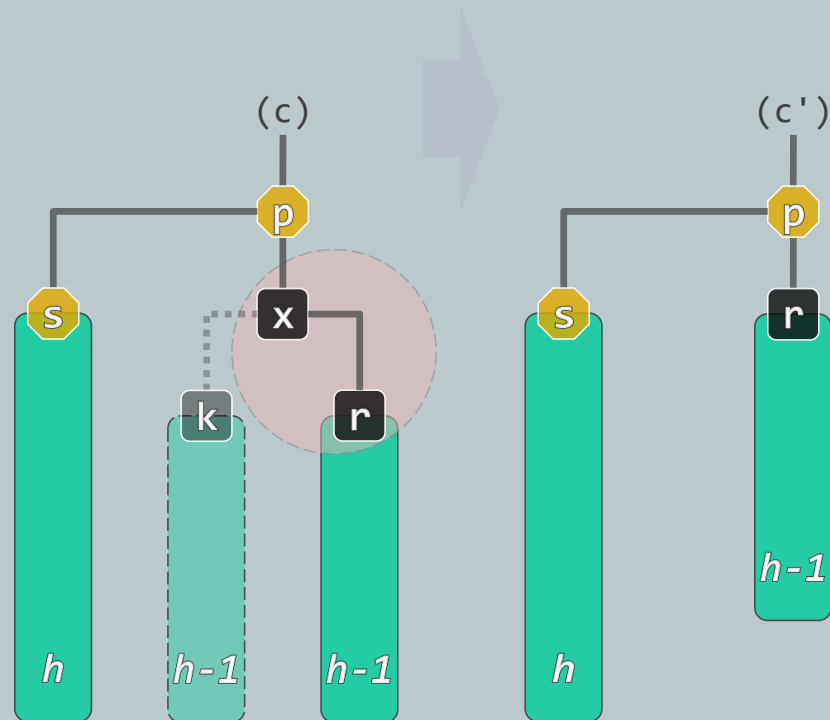
❖ 总之，无论x或r为红，则3、4均不难满足

——删除遂告完成！



双黑

- ❖ 若 x 与 r 均黑 (double black) , 则不然...
- ❖ 摘除 x 并代之以 r 后, 全树**黑深度**不再统一
(稍后可见, 等效于B-树中 x 所属节点**下溢**)
- ❖ 在新树中, 考查 r 的父亲、兄弟
 - $p = r \rightarrow \text{parent}$ //亦是原 x 的父亲
 - $s = \text{sibling}(r)$
- ❖ 以下分四种情况处理...



实现

```
template <typename T> bool RedBlack<T>::remove( const T & e ) {  
    BinNodePosi<T> & x = search( e ); if ( !x ) return false; //查找定位  
    BinNodePosi<T> r = removeAt( x, _hot ); //删除_hot的某孩子, r指向其接替者  
    if ( ! ( -- _size ) ) return true; //若删除后为空树, 可直接返回  
    if ( ! _hot ) { //若被删除的是根, 则  
        _root->color = RB_BLACK; //将其置黑, 并  
        updateHeight( _root ); //更新 (全树) 黑高度  
        return true;  
    } //至此, 原x (现r) 必非根
```

实现

❖ // 若父亲（及祖先）依然平衡，则无需调整

```
if ( BlackHeightUpdated( * _hot ) ) return true;
```

// 至此，必失衡

// 若替代节点r为红，则只需简单地翻转其颜色

```
if ( IsRed( r ) ) { r->color = RB_BLACK; r->height++; return true; }
```

// 至此，r以及被其替代的x均为黑色

```
solveDoubleBlack( r ); //双黑调整（入口处必有 r == NULL）
```

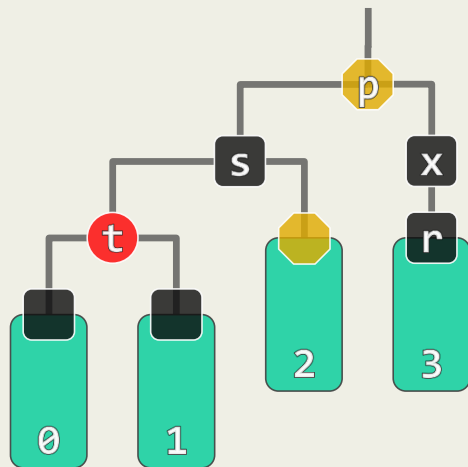
```
return true;
```

```
}
```

双黑修正

```
template <typename T> void RedBlack<T>::solveDoubleBlack( BinNodePosi<T> r ) {  
    BinNodePosi<T> p = r ? r->parent : _hot; if ( !p ) return; //r的父亲  
    BinNodePosi<T> s = (r == p->lc) ? p->rc : p->lc; //r的兄弟  
  
    if ( IsBlack( s ) ) { //兄弟s为黑  
        BinNodePosi<T> t = NULL; //s的红孩子（若左、右孩子皆红，左者优先；皆黑时为NULL）  
        if ( IsRed ( s->rc ) ) t = s->rc;  
        if ( IsRed ( s->lc ) ) t = s->lc;  
  
        if ( t ) { /* ... 黑s有红孩子: BB-1 ... */ }  
        else { /* ... 黑s无红孩子: BB-2R或BB-2B ... */ }  
    } else { /* ... 兄弟s为红: BB-3 ... */ }  
}
```

BB-1: s为黑, 且至少有一个红孩子t



❖ “3+4” 重构:

$t \sim a$

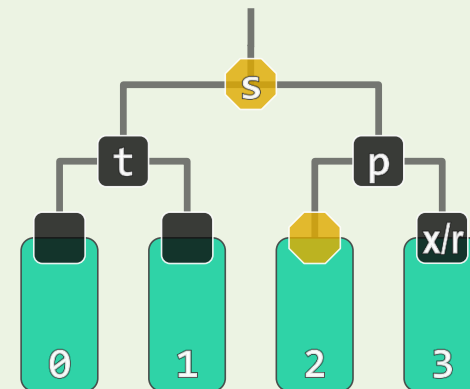
$s \sim b$

$p \sim c$

❖ r保持黑

a、c染黑

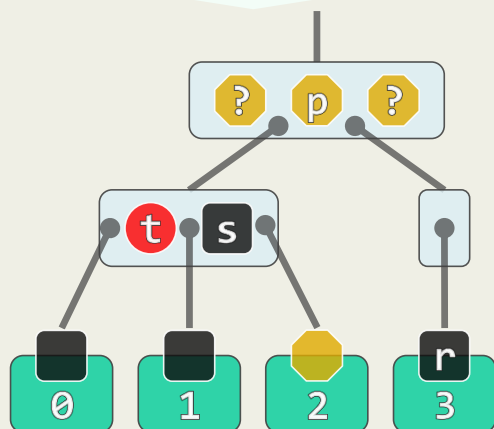
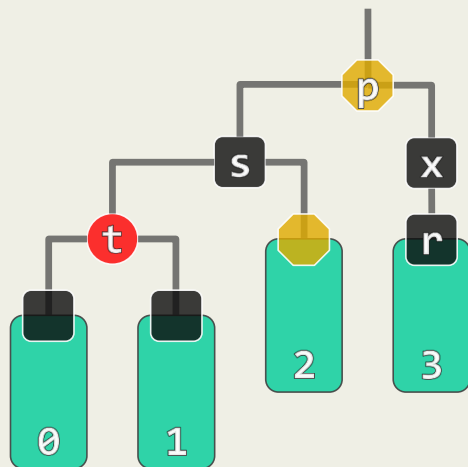
b继承p的原色



❖ 如此, 红黑树性质在全局得以恢复——删除完成! //zig-zag等类似

❖ 在对应的B-树中, 以上操作等效于...

BB-1: s为黑, 且至少有一个红孩子t



❖ 通过关键码的**旋转**
消除超级节点的下溢

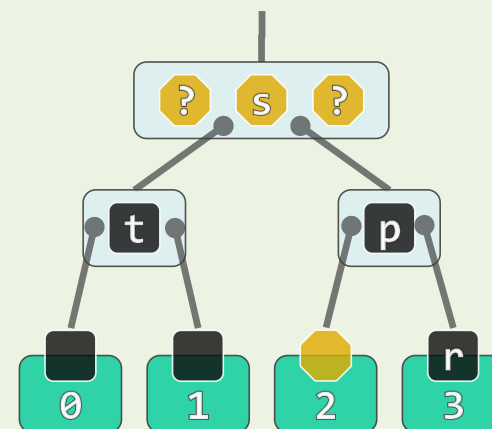
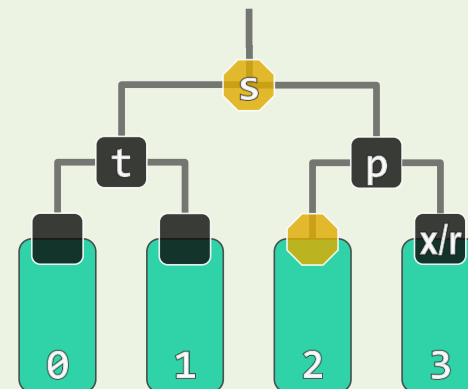
❖ 在对应的B-树中

- p若为红

问号之一为黑关键码

- p若为黑

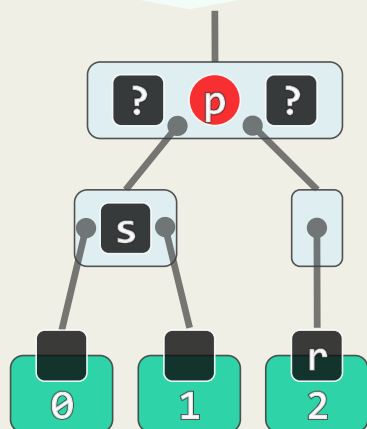
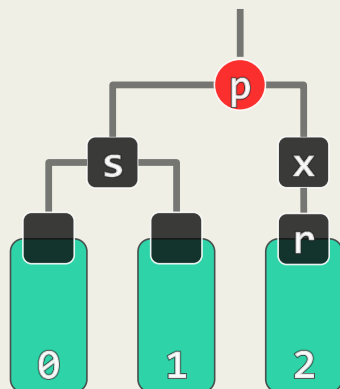
必自成一个超级节点



BB-1: 实现

```
if ( IsBlack( s ) ) { //兄弟s为黑
    /* ..... */
    if ( t ) { //黑s有红孩子: BB-1
        RBColor oldColor = p->color; //备份p颜色, 并对t、父亲、祖父
        BinNodePosi<T> b = FromParentTo( *p ) = rotateAt( t ); //旋转
        if ( HasLChild( *b ) ) { b->lc->color = RB_BLACK; updateHeight( b->lc ); }
        if ( HasRChild( *b ) ) { b->rc->color = RB_BLACK; updateHeight( b->rc ); }
        b->color = oldColor; updateHeight( b ); //新根继承原根的颜色
    } else { /* ... 黑s无红孩子: BB-2R或BB-2B ... */ }
} else { /* ... 兄弟s为红: BB-3 ... */ }
```

BB-2R: s为黑, 且两个孩子均为黑; p为红



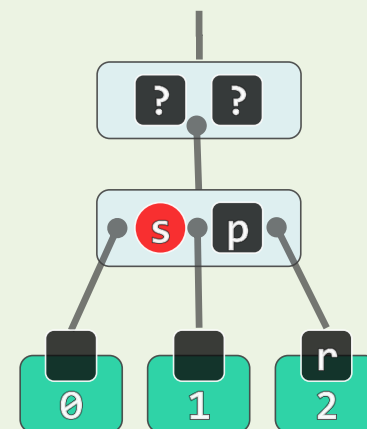
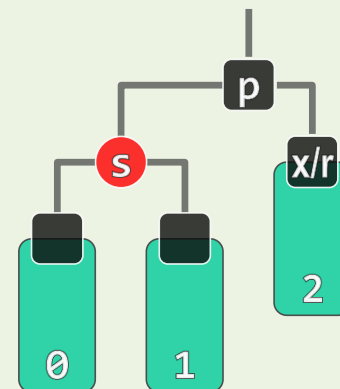
❖ r 保持黑; s 转红; p 转黑

❖ 在对应的B-树中, 等效于
下溢节点与兄弟合并

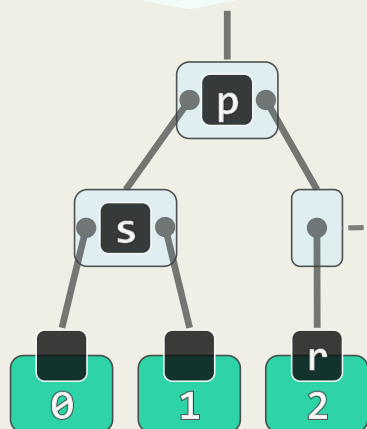
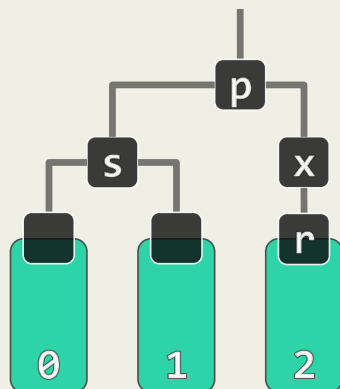
❖ 红黑树性质在**全局**得以恢复

❖ 失去关键码 p 后, 上层节点
会否**继而**下溢? 不会!

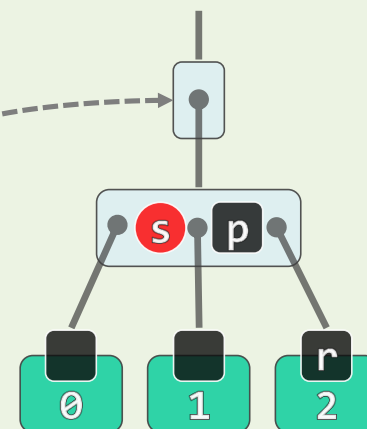
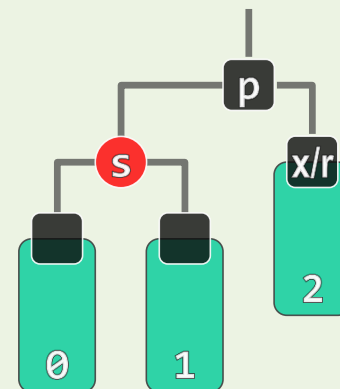
❖ 合并之前, 在 p 之左或右侧
还应有一个黑关键码



BB-2B: s为黑, 且两个孩子均为黑; p为黑



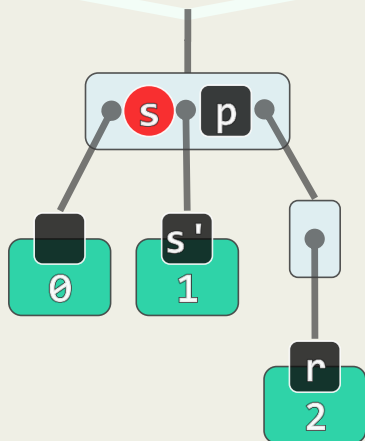
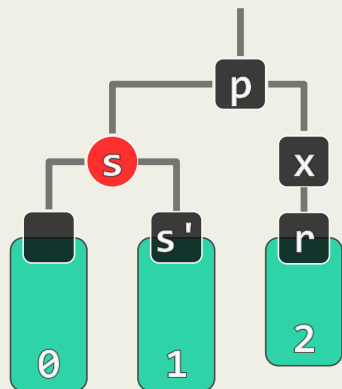
- ❖ s转红; r与p保持黑
- ❖ 红黑树性质在局部得以恢复
- ❖ 在对应的B-树中, 等效于
下溢节点与兄弟合并
- ❖ 合并前, p和s均属于单关键码节点
- ❖ 孩子的下溢修复后, 父节点继而下溢
- ❖ 好在可继续分情况处理
高度递增, 至多 $O(\log n)$ 层 (步)



BB-(2R+2B): 实现

```
if ( IsBlack( s ) ) { //兄弟s为黑
    /* ..... */
    if ( t ) { /* ... 黑s有红孩子: BB-1 ... */ }
    else { /* 黑s无红孩子 */
        s->color = RB_RED; s->height--; //s转红
        if ( IsRed( p ) ) //BB-2R: p转黑, 但黑高度不变
            { p->color = RB_BLACK; }
        else //BB-2B: p保持黑, 但黑高度下降; 递归修正
            { p->height--; solveDoubleBlack( p ); }
    }
} else { /* ... 兄弟s为红: BB-3 ... */ }
```

BB-3: s为红 (其孩子均为黑)



❖ 绕 p 单旋; s 红转黑, p 黑转红

❖ 黑高度依然异常, 但...

❖ r 有了一个新的黑兄弟 s'
故转化为前述情况, 而且...

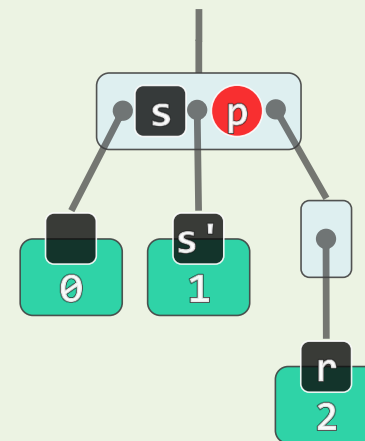
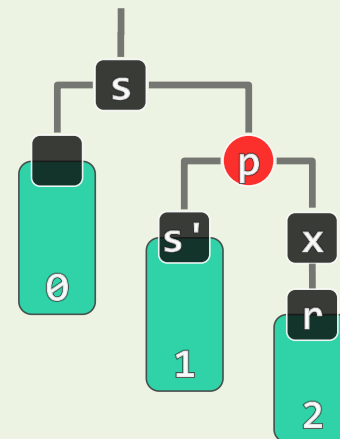
❖ 既然 p 已转红, 接下来

- 绝不会是BB-2B

- 而只能是BB-2R或BB-1

❖ 于是, 再经一轮调整

红黑树性质必然全局恢复



BB-3: 实现

```
❖ if ( IsBlack( s ) ) { //兄弟s为黑

    if ( t ) { /* ... 黑s有红孩子: BB-1 ... */ }

    else { /* ... 黑s无红孩子: BB-2R或BB-2B ... */ }

} else { //兄弟s为红: BB-3

    s->color = RB_BLACK; p->color = RB_RED; //s转黑, p转红

    BinNodePosi<T> t = IsLChild( *s ) ? s->lc : s->rc; //取t与其父s同侧

    _hot = p; FromParentTo( *p ) = rotateAt( t ); //对t及其父亲、祖父做平衡调整

    solveDoubleBlack( r ); //继续修正r——此时p已转红, 故后续只能是BB-1或BB-2R

}
```

复杂度

❖ `RedBlack<T>::remove()`

仅需 $O(\log n)$ 时间

- $O(\log n)$ 次重染色
- $O(1)$ 次旋转

	旋转	染色	此后
(1) 黑s有红子t	1~2	3	调整随即完成
(2R) 黑s无红子, p红	0	2	调整随即完成
(2B) 黑s无红子, p黑	0	1	必再次双黑, 但将上升一层
(3) 红s	1	2	转为(1)或(2R)

